JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class Instant

java.lang.Object
    java.time.Instant

**All Implemented Interfaces:**

Serializable, Comparable<Instant>, Temporal, TemporalAccessor, TemporalAdjuster

---

```
public final class Instant
extends Object
implements Temporal, TemporalAdjuster, Comparable<Instant>,
Serializable
```

An instantaneous point on the time-line.

This class models a single instantaneous point on the time-line. This might be used to record event time-stamps in the application.

The range of an instant requires the storage of a number larger than a `long`. To achieve this, the class stores a `long` representing epoch-seconds and an `int` representing nanosecond-of-second, which will always be between 0 and 999,999,999. The epoch-seconds are measured from the standard Java epoch of `1970-01-01T00:00:00Z` where instants after the epoch have positive values, and earlier instants have negative values. For both the epoch-second and nanosecond parts, a larger value is always later on the time-line than a smaller value.

## Time-scale

The length of the solar day is the standard way that humans measure time. This has traditionally been subdivided into 24 hours of 60 minutes of 60 seconds, forming an 86,400 second day.

Modern timekeeping is based on atomic clocks which precisely define an SI second relative to the transitions of a Caesium atom. The length of an SI second was defined to be very close to the 86,400th fraction of a day.

Unfortunately, as the Earth rotates the length of the day varies. In addition, over time the average length of the day is getting longer as the Earth slows. As a result, the length of a solar day in 2012 is slightly longer than 86,400 SI seconds. The actual length of any given day and the amount by which the Earth is slowing are not predictable and can only be determined by measurement. The UT1 time-scale captures the accurate length of day, but is only available some time after the day has completed.

The UTC time-scale is a standard approach to bundle up all the additional fractions of a second from UT1 into whole seconds, known as *leap-seconds*. A

leap-second may be added or removed depending on the Earth's rotational changes. As such, UTC permits a day to have 86,399 SI seconds or 86,401 SI seconds where necessary in order to keep the day aligned with the Sun.

The modern UTC time-scale was introduced in 1972, introducing the concept of whole leap-seconds. Between 1958 and 1972, the definition of UTC was complex, with minor sub-second leaps and alterations to the length of the notional second. As of 2012, discussions are underway to change the definition of UTC again, with the potential to remove leap seconds or introduce other changes.

Given the complexity of accurate timekeeping described above, this Java API defines its own time-scale, the *Java Time-Scale*.

The Java Time-Scale divides each calendar day into exactly 86,400 subdivisions, known as seconds. These seconds may differ from the SI second. It closely matches the de facto international civil time scale, the definition of which changes from time to time.

The Java Time-Scale has slightly different definitions for different segments of the time-line, each based on the consensus international time scale that is used as the basis for civil time. Whenever the internationally-agreed time scale is modified or replaced, a new segment of the Java Time-Scale must be defined for it. Each segment must meet these requirements:

- the Java Time-Scale shall closely match the underlying international civil time scale;
- the Java Time-Scale shall exactly match the international civil time scale at noon each day;
- the Java Time-Scale shall have a precisely-defined relationship to the international civil time scale.

There are currently, as of 2013, two segments in the Java time-scale.

For the segment from 1972-11-03 (exact boundary discussed below) until further notice, the consensus international time scale is UTC (with leap seconds). In this segment, the Java Time-Scale is identical to UTC-SLS. This is identical to UTC on days that do not have a leap second. On days that do have a leap second, the leap second is spread equally over the last 1000 seconds of the day, maintaining the appearance of exactly 86,400 seconds per day.

For the segment prior to 1972-11-03, extending back arbitrarily far, the consensus international time scale is defined to be UT1, applied proleptically, which is equivalent to the (mean) solar time on the prime meridian (Greenwich). In this segment, the Java Time-Scale is identical to the consensus international time scale. The exact boundary between the two segments is the instant where UT1 = UTC between 1972-11-03T00:00 and 1972-11-04T12:00.

Implementations of the Java time-scale using the JSR-310 API are not required to provide any clock that is sub-second accurate, or that progresses monotonically or smoothly. Implementations are therefore not required to actually perform the UTC-SLS slew or to otherwise be aware of leap seconds. JSR-310 does, however, require that implementations must document the approach they use when defining a clock representing the current instant. See `Clock` for details on the available clocks.

The Java time-scale is used for all date-time classes. This includes `Instant`, `LocalDate`, `LocalTime`, `OffsetDateTime`, `ZonedDateTime` and `Duration`.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

**Implementation Requirements:**

This class is immutable and thread-safe.

**Since:**

1.8

**See Also:**

Serialized Form

## *Field Summary*

**Fields**

| Modifier and Type | Field | Description |
|---|---|---|
| `static final Instant` | **`EPOCH`** | Constant for the 1970-01-01T00:00:00Z epoch instant. |
| `static final Instant` | **`MAX`** | The maximum supported `Instant`, '1000000000-12-31T23:59:59.999999999Z'. |
| `static final Instant` | **`MIN`** | The minimum supported `Instant`, '-1000000000-01-01T00:00Z'. |

## *Method Summary*

**All Methods** | **Static Methods**

| Modifier and Type | Method | Description |
| --- | --- | --- |
| Temporal | **adjustInto**(Temporal temporal) | Adjusts the specified temporal object to have this instant. |
| OffsetDateTime | **atOffset**(ZoneOffset offset) | Combines this instant with an offset to create an `OffsetDateTime`. |
| ZonedDateTime | **atZone**(ZoneId zone) | Combines this instant with a time-zone to create a `ZonedDateTime`. |
| int | **compareTo**(Instant otherInstant) | Compares this instant to the specified instant. |
| boolean | **equals**(Object other) | Checks if this instant is equal to the specified instant. |
| static Instant | **from**(TemporalAccessor tempor | Obtains an instance of `Instant` from a temporal object. |
| int | **get**(TemporalField field) | Gets the value of the specified field from this instant as an `int`. |
| long | **getEpochSecond**() | Gets the number of seconds from the Java epoch of 1970-01-01T00:00:00Z. |
| long | **getLong**(TemporalField field) | Gets the value of the specified field from this instant as a `long`. |
| int | **getNano**() | Gets the number of nanoseconds, later along the time-line, from the start of the second. |
| int | **hashCode**() | Returns a hash code for this instant. |
| boolean | | |

| | | |
|---|---|---|
| | **isAfter**<br>(Instant otherInstant) | Checks if this instant is after the specified instant. |
| boolean | **isBefore**<br>(Instant otherInstant) | Checks if this instant is before the specified instant. |
| boolean | **isSupported**<br>(TemporalField field) | Checks if the specified field is supported. |
| boolean | **isSupported**<br>(TemporalUnit unit) | Checks if the specified unit is supported. |
| Instant | **minus**<br>(long amountToSubtract, TemporalUnit unit) | Returns a copy of this instant with the specified amount subtracted. |
| Instant | **minus**<br>(TemporalAmount amountTo | Returns a copy of this instant with the specified amount subtracted. |
| Instant | **minusMillis**<br>(long millisToSubtract) | Returns a copy of this instant with the specified duration in milliseconds subtracted. |
| Instant | **minusNanos**<br>(long nanosToSubtract) | Returns a copy of this instant with the specified duration in nanoseconds subtracted. |
| Instant | **minusSeconds**<br>(long secondsToSubtract) | Returns a copy of this instant with the specified duration in seconds subtracted. |
| static Instant | **now**() | Obtains the current instant from the system clock. |
| static Instant | **now**(Clock clock) | Obtains the current instant from the specified clock. |

| | | |
|---|---|---|
| static Instant | **ofEpochMilli**<br>(long epochMilli) | Obtains an instance of Instant using milliseconds from the epoch of 1970-01-01T00:00:00Z. |
| static Instant | **ofEpochSecond**<br>(long epochSecond) | Obtains an instance of Instant using seconds from the epoch of 1970-01-01T00:00:00Z. |
| static Instant | **ofEpochSecond**<br>(long epochSecond,<br>long nanoAdjustment) | Obtains an instance of Instant using seconds from the epoch of 1970-01-01T00:00:00Z and nanosecond fraction of second. |
| static Instant | **parse**<br>(CharSequence text) | Obtains an instance of Instant from a text string such as 2007-12-03T10:15:30.00Z. |
| Instant | **plus**(long amountToAdd,<br>TemporalUnit unit) | Returns a copy of this instant with the specified amount added. |
| Instant | **plus**<br>(TemporalAmount amountTo | Returns a copy of this instant with the specified amount added. |
| Instant | **plusMillis**<br>(long millisToAdd) | Returns a copy of this instant with the specified duration in milliseconds added. |
| Instant | **plusNanos**<br>(long nanosToAdd) | Returns a copy of this instant with the specified duration in nanoseconds added. |
| Instant | **plusSaturating**<br>(Duration duration) | Returns a copy of this instant with the specified duration added, with saturated semantics. |

| | | |
|---|---|---|
| Instant | **plusSeconds**(long secondsToAdd) | Returns a copy of this instant with the specified duration in seconds added. |
| <R> R | **query**(TemporalQuery<R> query) | Queries this instant using the specified query. |
| ValueRange | **range**(TemporalField field) | Gets the range of valid values for the specified field. |
| long | **toEpochMilli**() | Converts this instant to the number of milliseconds from the epoch of 1970-01-01T00:00:00Z. |
| String | **toString**() | A string representation of this instant using ISO-8601 representation. |
| Instant | **truncatedTo**(TemporalUnit unit) | Returns a copy of this `Instant` truncated to the specified unit. |
| Duration | **until**(Instant endExclusive) | Calculates the `Duration` until another `Instant`. |
| long | **until**(Temporal endExclusive, TemporalUnit unit) | Calculates the amount of time until another instant in terms of the specified unit. |
| Instant | **with**(TemporalAdjuster adjust | Returns an adjusted copy of this instant. |
| Instant | **with**(TemporalField field, long newValue) | Returns a copy of this instant with the specified field set to a new value. |

**Methods declared in class Object**

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

## *Field Details*

### EPOCH

```
public static final Instant EPOCH
```

Constant for the 1970-01-01T00:00:00Z epoch instant.

### MIN

```
public static final Instant MIN
```

The minimum supported `Instant`, '-1000000000-01-01T00:00Z'. This could be used by an application as a "far past" instant.

This is one year earlier than the minimum `LocalDateTime`. This provides sufficient values to handle the range of `ZoneOffset` which affect the instant in addition to the local date-time. The value is also chosen such that the value of the year fits in an `int`.

### MAX

```
public static final Instant MAX
```

The maximum supported `Instant`, '1000000000-12-31T23:59:59.999999999Z'. This could be used by an application as a "far future" instant.

This is one year later than the maximum `LocalDateTime`. This provides sufficient values to handle the range of `ZoneOffset` which affect the instant in addition to the local date-time. The value is also chosen such that the value of the year fits in an `int`.

## *Method Details*

### now

```
public static Instant now()
```

Obtains the current instant from the system clock.

This will query the `system UTC clock` to obtain the current instant.

Using this method will prevent the ability to use an alternate time-source for testing because the clock is effectively hard-coded.

**Returns:**

the current instant using the system clock, not null

## now

```
public static Instant now(Clock clock)
```

Obtains the current instant from the specified clock.

This will query the specified clock to obtain the current time.

Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

**Parameters:**

`clock` - the clock to use, not null

**Returns:**

the current instant, not null

## ofEpochSecond

```
public static Instant ofEpochSecond(long epochSecond)
```

Obtains an instance of `Instant` using seconds from the epoch of 1970-01-01T00:00:00Z.

The nanosecond field is set to zero.

**Parameters:**

`epochSecond` - the number of seconds from 1970-01-01T00:00:00Z

**Returns:**

an instant, not null

**Throws:**

`DateTimeException` - if the instant exceeds the maximum or minimum instant

## ofEpochSecond

```
public static Instant ofEpochSecond(long epochSecond,
                                    long nanoAdjustment)
```

Obtains an instance of `Instant` using seconds from the epoch of 1970-01-01T00:00:00Z and nanosecond fraction of second.

This method allows an arbitrary number of nanoseconds to be passed in. The factory will alter the values of the second and nanosecond in order to ensure that the stored nanosecond is in the range 0 to 999,999,999. For example, the following will result in exactly the same instant:

```
Instant.ofEpochSecond(3, 1);
Instant.ofEpochSecond(4, -999_999_999);
Instant.ofEpochSecond(2, 1000_000_001);
```

**Parameters:**

`epochSecond` - the number of seconds from 1970-01-01T00:00:00Z

`nanoAdjustment` - the nanosecond adjustment to the number of seconds, positive or negative

**Returns:**

an instant, not null

**Throws:**

`DateTimeException` - if the instant exceeds the maximum or minimum instant

`ArithmeticException` - if numeric overflow occurs

## ofEpochMilli

`public static Instant ofEpochMilli(long epochMilli)`

Obtains an instance of `Instant` using milliseconds from the epoch of 1970-01-01T00:00:00Z.

The seconds and nanoseconds are extracted from the specified milliseconds.

**Parameters:**

`epochMilli` - the number of milliseconds from 1970-01-01T00:00:00Z

**Returns:**

an instant, not null

## from

`public static Instant from(TemporalAccessor temporal)`

Obtains an instance of `Instant` from a temporal object.

This obtains an instant based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `Instant`.

The conversion extracts the `INSTANT_SECONDS` and `NANO_OF_SECOND` fields.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `Instant::from`.

**Parameters:**

`temporal` - the temporal object to convert, not null

**Returns:**

the instant, not null

**Throws:**

`DateTimeException` - if unable to convert to an `Instant`

## parse

```
public static Instant parse(CharSequence text)
```

Obtains an instance of `Instant` from a text string such as `2007-12-03T10:15:30.00Z`.

The string must represent a valid instant in UTC and is parsed using `DateTimeFormatter.ISO_INSTANT`.

**Parameters:**

`text` - the text to parse, not null

**Returns:**

the parsed instant, not null

**Throws:**

`DateTimeParseException` - if the text cannot be parsed

## isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this instant can be queried for the specified field. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The supported fields are:

- `NANO_OF_SECOND`
- `MICRO_OF_SECOND`
- `MILLI_OF_SECOND`
- `INSTANT_SECONDS`

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

**Specified by:**

`isSupported` in interface `TemporalAccessor`

**Parameters:**

`field` - the field to check, null returns false

**Returns:**

true if the field is supported on this instant, false if not

## isSupported

`public boolean isSupported(TemporalUnit unit)`

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this date-time. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

If the unit is a `ChronoUnit` then the query is implemented here. The supported units are:

- `NANOS`
- `MICROS`
- `MILLIS`
- `SECONDS`
- `MINUTES`
- `HOURS`
- `HALF_DAYS`
- `DAYS`

All other `ChronoUnit` instances will return false.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

**Specified by:**

`isSupported` in interface `Temporal`

**Parameters:**

`unit` - the unit to check, null returns false

**Returns:**

true if the unit can be added/subtracted, false if not

## range

`public ValueRange range(TemporalField field)`

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This instant is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

**Specified by:**

`range` in interface `TemporalAccessor`

**Parameters:**

`field` - the field to query the range for, not null

**Returns:**

the range of valid values for the field, not null

**Throws:**

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

## get

`public int get(TemporalField field)`

Gets the value of the specified field from this instant as an `int`.

This queries this instant for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this date-time, except `INSTANT_SECONDS` which is too large to fit in an `int` and throws a

DateTimeException. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

**Specified by:**

get in interface TemporalAccessor

**Parameters:**

field - the field to get, not null

**Returns:**

the value for the field

**Throws:**

DateTimeException - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

## getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this instant as a long.

This queries this instant for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return valid values based on this date-time. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

**Specified by:**

getLong in interface TemporalAccessor

**Parameters:**

field - the field to get, not null

**Returns:**

the value for the field

**Throws:**

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

## getEpochSecond

```
public long getEpochSecond()
```

Gets the number of seconds from the Java epoch of 1970-01-01T00:00:00Z.

The epoch second count is a simple incrementing count of seconds where second 0 is 1970-01-01T00:00:00Z. The nanosecond part is returned by `getNano()`.

**Returns:**

the seconds from the epoch of 1970-01-01T00:00:00Z

## getNano

```
public int getNano()
```

Gets the number of nanoseconds, later along the time-line, from the start of the second.

The nanosecond-of-second value measures the total number of nanoseconds from the second returned by `getEpochSecond()`.

**Returns:**

the nanoseconds within the second, always positive, never exceeds 999,999,999

## with

```
public Instant with(TemporalAdjuster adjuster)
```

Returns an adjusted copy of this instant.

This returns an `Instant`, based on this one, with the instant adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

**Specified by:**

`with` in interface `Temporal`

**Parameters:**

`adjuster` - the adjuster to use, not null

**Returns:**

an `Instant` based on `this` with the adjustment made, not null

**Throws:**

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

## with

```
public Instant with(TemporalField field,
                    long newValue)
```

Returns a copy of this instant with the specified field set to a new value.

This returns an `Instant`, based on this one, with the value for the specified field changed. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the adjustment is implemented here. The supported fields behave as follows:

- `NANO_OF_SECOND` - Returns an `Instant` with the specified nano-of-second. The epoch-second will be unchanged.
- `MICRO_OF_SECOND` - Returns an `Instant` with the nano-of-second replaced by the specified micro-of-second multiplied by 1,000. The epoch-second will be unchanged.
- `MILLI_OF_SECOND` - Returns an `Instant` with the nano-of-second replaced by the specified milli-of-second multiplied by 1,000,000. The epoch-second will be unchanged.
- `INSTANT_SECONDS` - Returns an `Instant` with the specified epoch-second. The nano-of-second will be unchanged.

In all cases, if the new value is outside the valid range of values for the field then a `DateTimeException` will be thrown.

All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the

argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

**Specified by:**

`with` in interface `Temporal`

**Parameters:**

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

**Returns:**

an `Instant` based on `this` with the specified field set, not null

**Throws:**

`DateTimeException` - if the field cannot be set

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

## truncatedTo

`public Instant truncatedTo(TemporalUnit unit)`

Returns a copy of this `Instant` truncated to the specified unit.

Truncating the instant returns a copy of the original with fields smaller than the specified unit set to zero. The fields are calculated on the basis of using a UTC offset as seen in `toString`. For example, truncating with the `MINUTES` unit will round down to the nearest minute, setting the seconds and nanoseconds to zero.

The unit must have a duration that divides into the length of a standard day without remainder. This includes all supplied time units on `ChronoUnit` and `DAYS`. Other units throw an exception.

This instance is immutable and unaffected by this method call.

**Parameters:**

`unit` - the unit to truncate to, not null

**Returns:**

an `Instant` based on this instant with the time truncated, not null

**Throws:**

`DateTimeException` - if the unit is invalid for truncation

`UnsupportedTemporalTypeException` - if the unit is not supported

## plus

`public Instant plus(TemporalAmount amountToAdd)`

Returns a copy of this instant with the specified amount added.

This returns an `Instant`, based on this one, with the specified amount added. The amount is typically `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

**Specified by:**

`plus` in interface `Temporal`

**Parameters:**

`amountToAdd` - the amount to add, not null

**Returns:**

an `Instant` based on this instant with the addition made, not null

**Throws:**

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

## plusSaturating

`public Instant plusSaturating(Duration duration)`

Returns a copy of this instant with the specified duration added, with saturated semantics.

If the result is "earlier" than `MIN`, this method returns `MIN`. If the result is "later" than `MAX`, it returns `MAX`. Otherwise it returns `plus(duration)`.

**API Note:**

This method can be used to calculate a deadline from this instant and a timeout. Unlike `plus(duration)`, this method never throws `ArithmeticException` or `DateTimeException` due to numeric overflow or `Instant` range violation.

**Parameters:**

`duration` - the duration to add, not null

**Returns:**

an `Instant` based on this instant with the addition made, not null

**Since:**

26

## plus

```
public Instant plus(long amountToAdd,
                    TemporalUnit unit)
```

Returns a copy of this instant with the specified amount added.

This returns an `Instant`, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoUnit` then the addition is implemented here. The supported fields behave as follows:

- `NANOS` - Returns an `Instant` with the specified number of nanoseconds added. This is equivalent to `plusNanos(long)`.
- `MICROS` - Returns an `Instant` with the specified number of microseconds added. This is equivalent to `plusNanos(long)` with the amount multiplied by 1,000.
- `MILLIS` - Returns an `Instant` with the specified number of milliseconds added. This is equivalent to `plusNanos(long)` with the amount multiplied by 1,000,000.
- `SECONDS` - Returns an `Instant` with the specified number of seconds added. This is equivalent to `plusSeconds(long)`.
- `MINUTES` - Returns an `Instant` with the specified number of minutes added. This is equivalent to `plusSeconds(long)` with the amount multiplied by 60.
- `HOURS` - Returns an `Instant` with the specified number of hours added. This is equivalent to `plusSeconds(long)` with the amount multiplied by 3,600.
- `HALF_DAYS` - Returns an `Instant` with the specified number of half-days added. This is equivalent to `plusSeconds(long)` with the amount multiplied by 43,200 (12 hours).
- `DAYS` - Returns an `Instant` with the specified number of days added. This is equivalent to `plusSeconds(long)` with the amount multiplied by 86,400 (24 hours).

All other `ChronoUnit` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

**Specified by:**

`plus` in interface `Temporal`

**Parameters:**

`amountToAdd` - the amount of the unit to add to the result, may be negative

`unit` - the unit of the amount to add, not null

**Returns:**

an `Instant` based on this instant with the specified amount added, not null

**Throws:**

`DateTimeException` - if the addition cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

## plusSeconds

```
public Instant plusSeconds(long secondsToAdd)
```

Returns a copy of this instant with the specified duration in seconds added.

This instance is immutable and unaffected by this method call.

**Parameters:**

`secondsToAdd` - the seconds to add, positive or negative

**Returns:**

an `Instant` based on this instant with the specified seconds added, not null

**Throws:**

`DateTimeException` - if the result exceeds the maximum or minimum instant

`ArithmeticException` - if numeric overflow occurs

## plusMillis

```
public Instant plusMillis(long millisToAdd)
```

Returns a copy of this instant with the specified duration in milliseconds added.

This instance is immutable and unaffected by this method call.

**Parameters:**

`millisToAdd` - the milliseconds to add, positive or negative

**Returns:**

an `Instant` based on this instant with the specified milliseconds added, not null

**Throws:**

`DateTimeException` - if the result exceeds the maximum or minimum instant

`ArithmeticException` - if numeric overflow occurs

## plusNanos

```
public Instant plusNanos(long nanosToAdd)
```

Returns a copy of this instant with the specified duration in nanoseconds added.

This instance is immutable and unaffected by this method call.

**Parameters:**

`nanosToAdd` - the nanoseconds to add, positive or negative

**Returns:**

an `Instant` based on this instant with the specified nanoseconds added, not null

**Throws:**

`DateTimeException` - if the result exceeds the maximum or minimum instant

`ArithmeticException` - if numeric overflow occurs

## minus

```
public Instant minus(TemporalAmount amountToSubtract)
```

Returns a copy of this instant with the specified amount subtracted.

This returns an `Instant`, based on this one, with the specified amount subtracted. The amount is typically `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

**Specified by:**

`minus` in interface `Temporal`

**Parameters:**

`amountToSubtract` - the amount to subtract, not null

**Returns:**

an `Instant` based on this instant with the subtraction made, not null

**Throws:**

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

## minus

```
public Instant minus(long amountToSubtract,
                     TemporalUnit unit)
```

Returns a copy of this instant with the specified amount subtracted.

This returns an `Instant`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

**Specified by:**

`minus` in interface `Temporal`

**Parameters:**

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

**Returns:**

an `Instant` based on this instant with the specified amount subtracted, not null

**Throws:**

`DateTimeException` - if the subtraction cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

## minusSeconds

```
public Instant minusSeconds(long secondsToSubtract)
```

Returns a copy of this instant with the specified duration in seconds subtracted.

This instance is immutable and unaffected by this method call.

**Parameters:**

`secondsToSubtract` - the seconds to subtract, positive or negative

**Returns:**

an `Instant` based on this instant with the specified seconds subtracted, not null

**Throws:**

`DateTimeException` - if the result exceeds the maximum or minimum instant

`ArithmeticException` - if numeric overflow occurs

## minusMillis

```
public Instant minusMillis(long millisToSubtract)
```

Returns a copy of this instant with the specified duration in milliseconds subtracted.

This instance is immutable and unaffected by this method call.

**Parameters:**

`millisToSubtract` - the milliseconds to subtract, positive or negative

**Returns:**

an `Instant` based on this instant with the specified milliseconds subtracted, not null

**Throws:**

`DateTimeException` - if the result exceeds the maximum or minimum instant

`ArithmeticException` - if numeric overflow occurs

## minusNanos

```
public Instant minusNanos(long nanosToSubtract)
```

Returns a copy of this instant with the specified duration in nanoseconds subtracted.

This instance is immutable and unaffected by this method call.

**Parameters:**

`nanosToSubtract` - the nanoseconds to subtract, positive or negative

**Returns:**

an `Instant` based on this instant with the specified nanoseconds subtracted, not null

**Throws:**

`DateTimeException` - if the result exceeds the maximum or minimum instant

`ArithmeticException` - if numeric overflow occurs

## query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this instant using the specified query.

This queries this instant using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

**Specified by:**

`query` in interface `TemporalAccessor`

**Type Parameters:**

`R` - the type of the result

**Parameters:**

`query` - the query to invoke, not null

**Returns:**

the query result, null may be returned (defined by the query)

**Throws:**

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

## adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have this instant.

This returns a temporal object of the same observable type as the input with the instant changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` twice, passing `ChronoField.INSTANT_SECONDS` and `ChronoField.NANO_OF_SECOND` as the fields.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisInstant.adjustInto(temporal);
temporal = temporal.with(thisInstant);
```

This instance is immutable and unaffected by this method call.

**Specified by:**

`adjustInto` in interface `TemporalAdjuster`

**Parameters:**

`temporal` - the target object to be adjusted, not null

**Returns:**

the adjusted object, not null

**Throws:**

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

## until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another instant in terms of the specified unit.

This calculates the amount of time between two `Instant` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified instant. The result will be negative if the end is before the start. The calculation returns a whole number, representing the number of complete units between the two instants. The `Temporal` passed to this method is converted to a `Instant` using `from(TemporalAccessor)`. For example, the amount in seconds between two dates can be calculated using `startInstant.until(endInstant, SECONDS)`.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, SECONDS);
amount = SECONDS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `NANOS`, `MICROS`, `MILLIS`, `SECONDS`, `MINUTES`, `HOURS`, `HALF_DAYS` and `DAYS` are supported. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

**Specified by:**

`until` in interface `Temporal`

**Parameters:**

`endExclusive` - the end date, exclusive, which is converted to an `Instant`, not null

`unit` - the unit to measure the amount in, not null

**Returns:**

the amount of time between this instant and the end instant

**Throws:**

`DateTimeException` - if the amount cannot be calculated, or the end temporal cannot be converted to an `Instant`

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

## until

`public Duration until(Instant endExclusive)`

Calculates the `Duration` until another `Instant`.

The start and end points are `this` and the specified instant. The result will be negative if the end is before the start. Calling this method is equivalent to `Duration.between(this, endExclusive)`.

This instance is immutable and unaffected by this method call.

**Parameters:**

`endExclusive` - the end `Instant`, exclusive, not null

**Returns:**

the `Duration` from this `Instant` until the specified `endExclusive` `Instant`

**Since:**

23

**See Also:**

`Duration.between(Temporal, Temporal)`

## atOffset

`public OffsetDateTime atOffset(ZoneOffset offset)`

Combines this instant with an offset to create an `OffsetDateTime`.

This returns an `OffsetDateTime` formed from this instant at the specified offset from UTC/Greenwich. An exception will be thrown if the instant is too large to fit into an offset date-time.

This method is equivalent to `OffsetDateTime.ofInstant(this, offset)`.

**Parameters:**

`offset` - the offset to combine with, not null

**Returns:**

the offset date-time formed from this instant and the specified offset, not null

**Throws:**

`DateTimeException` - if the result exceeds the supported range

## atZone

`public ZonedDateTime atZone(ZoneId zone)`

Combines this instant with a time-zone to create a `ZonedDateTime`.

This returns an `ZonedDateTime` formed from this instant at the specified time-zone. An exception will be thrown if the instant is too large to fit into a zoned date-time.

This method is equivalent to `ZonedDateTime.ofInstant(this, zone)`.

**Parameters:**

`zone` - the zone to combine with, not null

**Returns:**

the zoned date-time formed from this instant and the specified zone, not null

**Throws:**

`DateTimeException` - if the result exceeds the supported range

## toEpochMilli

```
public long toEpochMilli()
```

Converts this instant to the number of milliseconds from the epoch of 1970-01-01T00:00:00Z.

If this instant represents a point on the time-line too far in the future or past to fit in a `long` milliseconds, then an exception is thrown.

If this instant has greater than millisecond precision, then the conversion will drop any excess precision information as though the amount in nanoseconds was subject to integer division by one million.

**Returns:**

the number of milliseconds since the epoch of 1970-01-01T00:00:00Z

**Throws:**

`ArithmeticException` - if numeric overflow occurs

## compareTo

```
public int compareTo(Instant otherInstant)
```

Compares this instant to the specified instant.

The comparison is based on the time-line position of the instants. It is "consistent with equals", as defined by `Comparable`.

**Specified by:**

`compareTo` in interface `Comparable<Instant>`

**Parameters:**

`otherInstant` - the other instant to compare to, not null

**Returns:**

the comparator value, that is less than zero if this instant is before `otherInstant`, zero if they are equal, or greater than zero if this instant is after `otherInstant`

**Throws:**

`NullPointerException` - if otherInstant is null

**See Also:**

`isBefore(Instant)`, `isAfter(Instant)`

## isAfter

```
public boolean isAfter(Instant otherInstant)
```

Checks if this instant is after the specified instant.

The comparison is based on the time-line position of the instants.

**Parameters:**

`otherInstant` - the other instant to compare to, not null

**Returns:**

true if this instant is after the specified instant

**Throws:**

`NullPointerException` - if otherInstant is null

## isBefore

```
public boolean isBefore(Instant otherInstant)
```

Checks if this instant is before the specified instant.

The comparison is based on the time-line position of the instants.

**Parameters:**

`otherInstant` - the other instant to compare to, not null

**Returns:**

true if this instant is before the specified instant

**Throws:**

`NullPointerException` - if otherInstant is null

## equals

```
public boolean equals(Object other)
```

Checks if this instant is equal to the specified instant.

The comparison is based on the time-line position of the instants.

**Overrides:**

`equals` in class `Object`

**Parameters:**

`other` - the other instant, null returns false

**Returns:**

true if the other instant is equal to this one

**See Also:**

`Object.hashCode()`, `HashMap`

## hashCode

```
public int hashCode()
```

Returns a hash code for this instant.

**Overrides:**

`hashCode` in class `Object`

**Returns:**

a suitable hash code

**See Also:**

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

## toString

```
public String toString()
```

A string representation of this instant using ISO-8601 representation.

The format used is the same as `DateTimeFormatter.ISO_INSTANT`.

**Overrides:**

`toString` in class `Object`

**Returns:**

an ISO-8601 representation of this instant, not null

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.